Review dossier L: logic

Reader edition of the work at commit 0f1d9aa13, 3 November 2026. Links into the repository are plain text here; the covering letter says how to reach it.

What this is. One part of the review packet of the Theory of Becoming, for readers in logic: relation algebra, coalgebra and bisimulation, constructive mathematics and modal logic. It sets out 29 items, each a definition or theorem of the formal root or of the logical method, and asks whether each is correctly stated, described in the chapters, credited where it restates known work, and stated at its exact strength.

Time, and the one-hour path. This front, to the first item, takes about ten minutes. Reading the whole with care takes about four hours (some 11,000 words of prose and 27 Lean blocks); an answer to every question would take several days. If you have one hour, skim **Terms** and **Reading the Lean blocks**, then give verdicts on these five, which only a logician can judge: L-06 (the core of a class as the greatest bisimulation inside it, and what its existence uses), L-09 (going on as a coinductive and ending as an inductive predicate), L-13 (the Hennessy–Milner converse in the grammar, refuted and proved under a premise), L-14 (its limit form and LLPO) and L-19 (what an empty axiom report shows). Questions marked (optional) are open research problems, not judgments asked of you.

How it was made. Everything in the theory was made with AI: the Lean proofs, the chapters, the readings of the classical texts and this dossier; no human has reviewed any of it. A script copied the Lean statements from the sources, and compared every block with them again for this version; the prose is AI-written and may be wrong. Re-read against the chapters and the Lean on 2 November 2026; an answered question or removed weakness is dated. Under **Known weaknesses**, a bullet citing a chapter or a review reports a weakness the work itself records; one marked [this dossier, AI] is this dossier's own assessment, for the reviewer to confirm or reject. The AI reviews of the chapters found one fault everywhere: "theorem sentences that drop an operative hypothesis, change a quantifier, turn a negative constructive conclusion into an existential one, or cite a neighbouring declaration of different strength" (a review record). The chapters were then corrected against the Lean; whether completely is part of what is asked.

What the theory says of itself. "None of the central results is new mathematics: each is near-definitional or a known result restated." (ch. 8) That verdict comes from an AI prior-art review of 20 October 2026 (BECOMING-SIGNIFICANCE-REVIEW), which read the ten central results of the earlier chapter before the grammar (Parts 1 and 2 below) existed; of the items here it covered only the premise ledger with its calibrations and the significance result of L-23. On the grammar, the judgments of prior art are the chapters' own credits at each result (01 §4 names bisimulation equivalence), the verdict of an AI review of 23 October comparing the grammar with the two design notes it was drafted from, not with the classical texts (review of 23 October 2026; L-01), and the credits this dossier states at each item, which the reviewer is asked to check.

Terms. Lean paths are relative to `experiments/integrated-correction/general-framework/generative-basis/lean/becoming/` unless they begin otherwise.

- *The grammar* (the theory's *root*): the modules `Becoming/Grammar*.lean`, a calculus of relations (Part 1). A *root theorem* holds for every carrier, relation and class; a *witness* is a single constructed model.
- *The pure core*: the modules of the Lean package `Becoming` that import only three foundation modules and one another and compile from Lean's prelude (`Init`) alone; every source declaration in them has an empty axiom report (L-19). The *periphery* is everything outside it; it may use Mathlib and classical

axioms, each reported. *Older modules* in `lean/deeper/` predate the package and are checked on their own, outside it. Not to be confused with *the core of a class* (`«核»`, L-06).

- *Models*. A *process* (`Process S A`) is a labelled step relation `S → A → S → Prop` on configurations; a *frame* adds claims realized at configurations. The *World* is the richest model: relata that take from and give to one another's natural-number levels across discrete steps, each active or not, under eight named premises (balance, a census of what is active, and others) that a world may meet or fail (01 §8).
- *Calibration*: a law proved equivalent to, or conditional on, an omniscience principle. By the theory's own criterion (08 §5) it is a result only where it says what the need means for the unfolding: "Only a calibration that says something of that kind is a result. One that concerns the logic alone is a lemma of its law." (ch. 8)
- *Record*: the work's entry for one result (statement, Lean declarations, premises, what it does not show), linked from the chapters.
- *Readings and classes*. [AI reading, Dn] marks an interpretation made by AI, `Dn` its entry in the work's register of decisions; a reading changes no theorem. A quoted sentence of the classical texts, given with its unit's id, carries the class an AI coder gave it, glossed with the names; its English translation is this dossier's (AI).

Reading the Lean blocks. Each block is Lean 4 source copied from the file named under it; theorem statements stop where the proof begins (`:=`), definitions are given whole.

- `def` defines a term; `abbrev` is a definition Lean unfolds freely; `structure … : Prop` is a conjunction with named fields; `inductive … → Prop` is the least predicate closed under the listed constructors (so `«及»` is the transitive closure and `«尽»` an accessibility predicate); `theorem name hypotheses : statement` states a theorem.
- Arguments in `( )` are explicit, in `{ }` implicit (inferred where the theorem is used), in `[ ]` instances such as `[DecidableEq S]`. A type variable such as `α`, declared once in the module (`variable {α : Type u}`), is universally quantified: a grammar theorem holds for every carrier.
- `Prop` is the type of propositions. Excluded middle is never assumed for all propositions: `P ∨ ¬ P` is proved for particular ones (for instance where `P` is decidable, or is a Boolean equation `b = true`) or taken as a hypothesis, as `em : ∀ P : Prop, P ∨ ¬ P` is in L-10. The theory writes `Decided p` for `p ∨ ¬ p` (a proposition); `Decidable p` is a procedure deciding `p` (data, stronger); `Nonempty T` says that the type `T` has an element.
- `fun x y => …` is λ-abstraction and application is juxtaposition (`R x y`); `.root` or `.u` names a constructor of a type fixed by the context.

Names. Two tables at the end gloss the vocabulary and the theorem names word for word (the names); keep them open beside the items.

Part 1. The grammar

L-01 The calculus and its operations

```
abbrev «际» (α : Type u) : Type u := α → α → Prop

def «而» (R S : «际» α) : «际» α := fun x z => ∃ y, R x y ∧ S y z

def «非» (R : «际» α) : «际» α := fun x y => ¬ R x y

inductive «及» (R : «际» α) : «际» α
  | one {x y : α} : R x y → «及» R x y
  | more {x y z : α} : R x y → «及» R y z → «及» R x z

structure «类» (E : «际» α) : Prop where
  refl : ∀ x, E x x
  symm : ∀ {x y}, E x y → E y x
  trans : ∀ {x y z}, E x y → E y z → E x z
```

File: `Becoming/Grammar.lean`. **Meaning.** A relation is a Prop-valued two-place predicate on a type parameter; composition, converse, meet and complement are defined pointwise; reach is the inductive transitive (not reflexive) closure; a class is an equivalence relation. Theorems are inclusions or pointwise equivalences, never equations between relations, so neither propositional nor function extensionality is used. Equality on the carrier is declared a commitment of the model. The operations are those of Tarski's calculus of relations (Tarski 1941; Tarski and Givant 1987), with four departures: there is no join (union), and with intuitionistic complement it is not recovered from meet and complement (`¬(¬R ⊓ ¬S)` gives only the double negation of the join); complement is intuitionistic; reach (not expressible in Tarski's calculus) is added; and the core (L-06) quantifies over all relations. **Standing and use.** Definitions; the base of every root theorem (01 §1). The chapter: "Every result is pointwise, so no extensionality enters, and the grammar imports only Lean's prelude." (ch. 1) **Reading and class.** The Chinese names are read from classical sentence forms (06); the operations are not drawn from the texts, and no corpus unit is classified as this definition.

Known weaknesses.

- The review of the grammar against the design notes it was drafted from: "Tarski's relation algebra with Chinese names." (the review of the grammar against its sources)
- [this dossier, AI] The module header calls the calculus point-free; the operations have pointwise semantics, as relation algebra has in set relations, and no term syntax is fixed, so which expressions make up the point-free calculus is not stated.
- [this dossier, AI] With reach and quantification over relations, the method chapter's literature sentence, "A pure relational calculus is as strong as three-variable first-order logic." (ch. 8) does not describe this calculus. 08 §6 says of that sentence and of its companions on pairing and power: "These are known results from the literature, not checked here." (ch. 8)
- What the calculus is said not to state (01, Open). A rule that changes by a fixed law is stated by taking the rule into the occurrence, the carrier being a parameter; "The law of change stays fixed, and the levels come from the carrier's product, not from the grammar." (ch. 1) A tower of rules with no top is stated, on processes, as one process with a fixed scheme (07 §2). Not stated: a change of the law of

change with no fixed scheme, or a space of rules not given with the carrier; a pattern reshaping its own limits; selection of selection as a law relating two grains. Limit (限) is defined nowhere, nor the conflict relation of a net of possibilities, which the module header mentions.

- [this dossier, AI] These are missing constructions, not shown limits of expressiveness: the relation is fixed, the carrier unrestricted. `continuationStep` encodes a time-varying law `R : Nat → F → F → Prop` as one relation on `Nat × F`, and an older module outside the package (`historyProcess`, in `lean/deeper/HistoryRealization.lean`) encodes history dependence as steps between complete histories. Missing are native operators and, beyond two models, representation theorems: since 29 October 2026 processes are represented both ways, and the World both ways for its relations and not on its amounts (L-21). A non-expressibility claim would need an object language, admissible encodings and a preservation criterion, none of which is fixed.

Questions.

1. Which expressions would form the point-free syntax the header names, and what is the accurate name for the resulting calculus?
2. (optional) Which laws of relation algebra (Tarski's axioms, the Schröder rules, the modular law) hold for these operations with intuitionistic complement, which makes the algebra not Boolean (none are listed anywhere), and should the theory state them?
3. (optional) With transitive closure and quantification over relations, where does the calculus sit in expressive strength (relation algebra with star, Freyd and Scedrov's allegories, higher-order logic), and against which object language should the missing constructions be judged?

L-02 Reach, precedence and co-presence

```
theorem «缘先果后当且仅当无环» (R : «际» α) : (∀ x y, R x y → ¬ «及» R y x) ↔ «无环» R

def «俱有» (R : «际» α) : «际» α := fun x y => «及» R x y ∧ «及» R y x

theorem «俱有为一念» (R : «际» α) :
    (∀ {x y}, «俱有» R x y → «俱有» R y x) ∧
      (∀ {x y z}, «俱有» R x y → «俱有» R y z → «俱有» R x z) ∧
      (∀ {x y}, «俱有» R x y → «俱有» R x x)

theorem reach_links_all_iff_em :
    (∀ c : Causet, c.Ordered → ∀ i j, c.prec i j → «及» (links c.prec) i j) ↔
      ∀ P : Prop, P ∨ ¬ P
```

Files: `Becoming/Grammar.lean`, `Becoming/GrammarCandidates.lean`, `Becoming/CausalGrowth.lean` (namespace `CausalGrowth`), `Becoming/Presence.lean` (namespace `Presence`). **Meaning.** Without cycles reach is a strict order (`«无环则及为序»`); every condition strictly precedes what it conditions exactly where there is no cycle; mutual conditions are co-present (`«相由则俱有»`), and co-presence is symmetric, transitive, and reflexive on what it relates. These are standard graph facts: an acyclic relation's transitive closure is a strict order; mutual reach relates nodes on a cycle within one strongly connected component. On precedence as causal set theory takes it (since 28 October 2026): a link of `«及» R`, a pair it relates with nothing it relates between, is a step of `R`; where each element is preceded only by smaller labels and precedence is decided and transitive, precedence is exactly the reach of its links; deciding what lies between two elements suffices, and that every pair of every causal set (transitive, naturally labelled, finite,

precedence not decided) is reached by its links is exactly excluded middle (`CausalGrowth.reach_links_all_iff_em`, above). On `ω + 1` no chain of links reaches the top (`OmegaPlusOne.not_reach_top`). **Standing and use.** Root theorems, and on orders of the numbers a calibration and a witness; the order of time is taken to be reach (01 §1). **Reading and class.** "緣成由於果，以果後、緣先故。" (ZL-01-132) (*a condition becomes a condition through its result, since the result comes after and the condition before*; Qingmu's commentary on the 中论) classified 乙 and proved under the premise that the net has no cycle; "雖則九世有隔不同，相由成立；" (JSZ-P-07-029) (*though the nine times are apart and different, they are established through one another*) and "融通無礙，同為一念。" (JSZ-P-07-030) (*they interpenetrate without obstruction and are together one thought-moment*; Fazang's 金师子章) classified 乙 and proved. The readings of these lines are for the Buddhist dossier.

Known weaknesses.

- Time and the four marks separate two occurrences alike in every relation only by equality on the carrier; the chapter says so: "a stationary process has two moments alike in every relation, which nothing but that equality tells apart." (ch. 1) The witness (the integers under successor, whose `0` and `1` are related by the two-way core of the full relation, in the script's own copy of `«位核»`, while `0` reaches `1`) is checked only in a script outside the package (theorem stationary_one_place of `_process/research/2026-10-05-generative-foundation-review/root-foundation/development/one-unfolding-root/review-checks/V9.lean`), not in the pure core.
- Co-presence is mutual reach, not presence in time; the grammar has no relation of presence in time, and adding one is a choice of model, so whether a condition must still be there when what it conditions arises is not expressible (01, Open). Since 28 October 2026 one model answers that question: on causal-set growth, the present read as the maximal elements of what a course has born, a condition is present at the birth of what it conditions on some course exactly at a link, precedence weakly decided (`Presence.present_some_iff_link`), and on every course exactly at the one link (`Presence.present_every_iff_only_link`; 01 §1). That answer rests on this reading of the present, other readings not varying with the course, and on the model's premise that labels carry no physical meaning; it adds no relation to the grammar.

Questions.

1. Is taking precedence as the transitive closure of an arbitrary condition relation, strict only where acyclic, a sound basis for an order of time (compare causal sets, which assume acyclicity)?
2. Is treating equality on the carrier as a commitment of the model a coherent stance in a type theory where every type carries an equality?

L-03 Continuation as a meet of a relation and an equivalence

```
def «续» (R E : «际» α) : «际» α := «合» R E

theorem «任一缘可为续» {R : «际» α} {x y : α} (h : R x y) :
    ∃ E : «际» α, «类» E ∧ «续» R E x y

theorem «分叉» :
    ∃ (α : Type) (R : «际» α) (a b c : α),
      «相续» R «—» a b ∧ «相续» R «—» a c ∧ ¬ «相续» R «—» b c ∧ ¬ «相续» R «—» c b ∧ b ≠ c
```

File: `Becoming/Grammar.lean`. **Meaning.** `y` continues `x` under `E` where `R x y` and `E x y`. Coarsening the class keeps every continuation (`«粗化»`); every single condition `R x y` is a continuation under the class relating `a` and `b` where `(a = x ∨ a = y) ↔ (b = x ∨ b = y)` (classically, a partition into at most two blocks, the ends and the rest; constructively, membership in the ends is not decided); two conditions of one occurrence, each alike to it under the class, are both continued by it (`«蜡烛之合»`); continuation under the full class can fork so that line-sameness is not transitive. **Standing and use.** The centre of the root, used by every later chapter (01 §2). **Reading and class.** [AI reading, D69]: the class is plural, after "有以同，類同也。" (MJ-S-089.4) (*being alike in some respect is likeness of kind*; 墨经·经说上; classified 甲). The dilemma "若一則無緣，若異則無相續，後當種種破，是故復說不一不異。" (ZL-01-031) (*if one, there is no condition; if different, there is no continuity; both are refuted in many ways below, and so it says again: neither one nor different*; Qingmu's commentary) is classified 己, a contrary reading; it is the Buddhist dossier's.

Known weaknesses.

- Containment in the condition relation is necessary only: by that class, continuation cannot by itself tell a fire passed on from one fire ending and another lighting (01, does not show).

Questions.

1. Is the meet `R ⊓ E` the natural formal choice for *conditioned and alike*, against alternatives such as `E;R;E` or a named sub-relation of `R`?
2. Does this construction show that the admissible classes must be restricted (for instance, to classes definable without parameters), and is that a known problem for criteria of persistence?
3. Is this construction known, for example as genidentity (Lewin 1922; on Lewin's and Reichenbach's uses, Padovani 2013) or a relational form of it?

L-04 The four marks and coarsening

```
def «生» (R E : «际» α) (y : α) : Prop := ¬ «有前» R E y

def «住» (R E : «际» α) (x : α) : Prop := «有前» R E x ∧ «有后» R E x

theorem «四相无自立» {R : «际» α} (irr : ∀ z, ¬ R z z) {x y z : α} (hxy : R x y) (hyz : R y z) :
    «住» R «一» y ∧ «生» R «静» y ∧ «亡» R «静» y

theorem «两端分则中有标» {R E K P : «际» α} (split : ∀ x y, «续» R E x y → K x y ∨ P x y)
    (trans : ∀ {a b c}, K a b → K b c → K a c) {x y : α} (line : «相续» R E x y) :
    ¬ K x y → ∃ u v, «续» R E u v ∧ P u v ∧ (x = u ∨ «相续» R E x u) ∧ (v = y ∨ «相续» R E v y)
```

File: `Becoming/Grammar.lean`. **Meaning.** Under a class, an occurrence begins where it has no predecessor under continuation, ends where it has no successor, and abides where it has both a predecessor and a successor. With `R` irreflexive, a middle occurrence abides under the full class and both begins and ends under identity. Ending under a coarser class implies ending under a finer one (`«亡于粗则亡于细»`); a beginning under a finer class continued under a coarser one is an alteration (`«生于细而续于粗则异»`); for a transitive relation `K` (a class is one), if every step either keeps `K` or carries `P`, a line whose ends `K` separates (`¬ K x y`) has a step carrying `P`. **Standing and use.** Root theorems (01 §3). **Reading and class.** Not a corpus unit; the readings of the four marks (四相) are the Buddhist dossier's.

Known weaknesses.

- The marks are one-step notions: a flash both begins and ends, so they are no partition even where decided (01 §3 says so).
- The definition of `«异»` does not require the second class to be finer than the first; 01 §3 says only that in use it is.

Questions.

1. Is `«四相无自立»` correctly described by the chapter's statement (irreflexive `R`, the full class and identity) and by its gloss?
2. Are one-step predecessor and successor an adequate formalization of beginning and ending, given cycles and two-way infinite lines, where nothing begins or ends?
3. Is `«两端分则中有标»` a known discrete intermediate-value form, and should it be credited?

L-05 Constancy never lets a continuation end

```
def «有常» (R E : «际» α) : Prop := «则» («而» («逆» E) R) («而» R («逆» E))

theorem «有常则续不尽» {R E : «际» α} (h : «有常» R E) {y : α} (hy : «有前» R E y) : «有后» R E y

theorem «有亡之续则不有常» {R E : «际» α} {x y : α} (hxy : «续» R E x y) (hy : «亡» R E y) :
    ¬ «有常» R E
```

File: `Becoming/Grammar.lean`. **Meaning.** `E` is constant under `R` where it is a one-way simulation: an `E`-mate of a node matches each of its steps by a step to an `E`-mate (`«有常之逐点»`). Then whatever continues something is continued, for every relation `E`, symmetric or not; a class under which some line ends is not constant. This is the standard fact that a simulation transfers the ability to step (Park 1981; Milner 1989), applied as the first weakness says. **Standing and use.** Root theorems (01 §4); they move the regularity of an ending line from its class to its core. **Reading and class.** None.

Known weaknesses.

- The proof applies constancy to the pair `(x, y)` itself, which is both `E`-related and `R`-related: the theorem says that a simulation relating a node to its successor propagates along the line.
- Constancy warrants no class (01, does not show): "identity is constant under every relation" (ch. 1) (`«静有常»`). A class's being constant says only that it agrees with the relation, and the finest class always qualifies; this bears on Q13 and on the reading of the core (L-06).

Questions.

1. Is that description of the theorem and its proof accurate?
2. Does the name *constancy* fit a condition that no class of an ending line meets and that identity always meets?
3. (optional) Would a weaker notion (simulation restricted to pairs not related by `R`, or relative to a sub-relation) capture regularity of patterns that end, and is such a notion known?

L-06 The root theorem: the core

```
def «核» (R E : «际» α) : «际» α :=
  fun u u' => ∃ S : «际» α, «则» S E ∧ «则» S («逆» S) ∧ «有常» R S ∧ S u u'

theorem «根» (R E : «际» α) :
    «则» («核» R E) E ∧ «有常» R («核» R E) ∧ «则» («核» R E) («逆» («核» R E)) ∧
      ∀ S, «则» S E → «则» S («逆» S) → «有常» R S → «则» S («核» R E)

theorem «类有常当且仅当含于核» (R : «际» α) {E : «际» α} (hE : «类» E) :
    «有常» R E ↔ «则» E («核» R E)

theorem «定后遍后则核为其内最粗同行之类» {R E : «际» α} (hE : «类» E) (fixed : «定后» R)
    (total : ∀ x, ∃ y, R x y) :
    («类» («核» R E) ∧ «则» («核» R E) E ∧ «同行» R («核» R E)) ∧
      ∀ S, «类» S → «则» S E → «同行» R S → «则» S («核» R E)

theorem «后不定则核可不同行» : ∃ (R E : «际» Nat),
    R 0 1 ∧ R 0 2 ∧ (∀ x y, E x y ↔ x = y) ∧
      «类» E ∧ ¬ «定后» R ∧ «核» R E 0 0 ∧ ¬ «核» R E 1 2 ∧ ¬ «同行» R («核» R E)
```

Files: `Becoming/Grammar.lean`, `Becoming/GrammarLaws.lean`. **Meaning.** The core of `E` under `R` is the union of all symmetric relations inside `E` that are constant under `R`; it lies inside `E`, is constant and symmetric, and contains every such relation. For a class, the core is a class (`«核为类»`), and the class is constant exactly where it lies inside its core. This is the greatest bisimulation inside `E` (inside `E ∩ E˘` where `E` is not symmetric), built as the union of bisimulations, the last conjunct of `«根»` being the coinduction principle (Park 1981; Milner 1989; coalgebraically, Rutten 2000); for a class it is, classically, the coarsest stable refinement of `E` (Paige and Tarjan 1987; Kanellakis and Smolka 1990). Since 31 October 2026 the pure core states one case of that: where the step's successor is fixed (`«定后»`, `R˘;R ⊆ 静`) and every occurrence has one, the core is the coarsest class inside `E` that the step goes alike across (`«同行»`, `R˘;E;R ⊆ E`). With no fixed successor the core need not go alike across the step (`«后不定则核可不同行»`). **Standing and use.** Root theorem; the core is what testing approximates (L-12), what the flame witnesses of 01 §5 use and, for a determined law read through a map, what knowing's 文 (a course read through a map) instantiates (01 §4). **Reading and class.** [AI reading, D65]: "The core is what continues of its own, the regularity a line carries by itself." (ch. 1) 01 §4 reads the core of a class as a 理 where the successor is fixed (`«定后则类之核同行»`), and as the 理 of `E`, the coarsest, where every occurrence also has one (the theorem above). [this dossier, AI] Off a fixed successor these theorems do not back that reading, since the core need not go alike. Not a corpus unit.

Known weaknesses.

- [this dossier, AI] At the theorem the chapter credits no prior result, "So the core is the greatest matching relation within the class, and it always exists." (ch. 1) The credit above is this dossier's.
- [this dossier, AI] The core is defined by an existential over all relations inside `Prop`, so its existence uses Lean's impredicative `Prop`; the pure-core discipline (L-19) reports axioms, not this.
- In one flame the witnesses that use the core single out nothing (01 §5): the joint class of its two descriptions is identity on the four moments, so each suffices for the other's regularity, and since neither is constant (a line under each ends) the two need each other through the core (`«火薪相须»`), and every description that with "burning or not" tells the moments apart suffices (`«火之见证不择类»`). In

two flames side by side the fuel left suffices while sameness of flame does not (`«两火以薪定理»`, `«两火同火不足以定理»`): a decision against one named alternative, not in general.

Questions.

1. Is the credit above accurate and sufficient, and is the restriction to `E ∩ E˘` stated correctly?
2. Does the existence of the core for every relation and class depend on impredicativity, and should that be stated as a commitment in a predicative or coinductive setting?
3. Does the mathematics support the D65 reading of the core?

L-07 Closure and bisimulation equivalence

```
theorem «闭合当且仅当有常» (R : «际» α) {E : «际» α} (hE : «类» E) :
    «有常» R E ↔ «则» («而» («而» E R) E) («而» R E)

theorem alike_iff_core {next : S → S} {o : S → R} {s s' : S} :
    Alike (frameBy next o) (frameBy next o) s s' ↔ «核» (step next) (sameValue o) s s'

theorem core_iff_same_course {p : Nat → Prop} {s s' : Nat} :
    «核» Steps.next (Steps.alike p) s s' ↔ ∀ n, (p (s + n) ↔ p (s' + n))
```

Files: `Becoming/Grammar.lean`, `Becoming/GrammarCourse.lean`, `Becoming/Change.lean`. **Meaning.** For a class, constancy is exactly `E;R;E ⊆ R;E`: the stability of the partition `E` under `R` of partition refinement (Kanellakis and Smolka 1990; Paige and Tarjan 1987), that is, `E` is a bisimulation equivalence, as the module credits (Park, Milner). For a determined law read through a map, bisimilarity is the core of the value class (`Law.alike_iff_core`); along the steps, the core of *alike in* `p` relates two steps where `p` agrees at every pair of steps equally far after them. **Standing and use.** Root theorem and two instances; the chapter: "For a class this is bisimulation equivalence, with the lumpability of Markov chains and computational closure as analogues not proved here." (ch. 1) (01 §4). **Reading and class.** None.

Known weaknesses.

- For branching processes no theorem joins likeness by bisimulation (01 §9.2) to the core: the only theorem joining them is for determined laws read through a map (`Law.alike_iff_core`), while the chapter says "On the models the core is the familiar invariant." (ch. 1)
- Lumpability is probabilistic and computational closure concerns ε-machines. Lumpability (Kemeny and Snell 1960; as probabilistic bisimulation, Larsen and Skou 1991) asks equal transition-probability totals from a block's members into each block, which nothing here expresses; both stay analogies, as the chapter says.

Questions.

1. In an intuitionistic setting, where a block's lying inside or outside a preimage is not decided, is `E;R;E ⊆ R;E` the right form of stability?
2. Should the correspondence for branching processes be proved before the chapter calls the core the familiar invariant on the models?
3. Is the chapter's wording of the two analogies accurate?

L-08 The two-way core and positions

```
def «位核» (R E : «际» α) : «际» α :=
  fun u u' => ∃ S : «际» α, «则» S E ∧ «则» S («逆» S) ∧ «有常» R S ∧ «有常» («逆» R) S ∧ S u u'

theorem «位同则生亡同» {R E : «际» α} (hE : «类» E) {y y' : α} (hk : «位核» R E y y') :
    («生» R E y → «生» R E y') ∧ («亡» R E y → «亡» R E y')

theorem «位同则住同» {R E : «际» α} (hE : «类» E) {y y' : α} (hk : «位核» R E y y')
    (hy : «住» R E y) : «住» R E y'

theorem «生于逆核下不变» {R E : «际» α} (hE : «类» E) {y y' : α} (hk : «核» («逆» R) E y y')
    (hy : «生» R E y) : «生» R E y'
```

Files: `Becoming/Grammar.lean`, `Becoming/GrammarConditionals.lean`. **Meaning.** The two-way core asks constancy under `R` and its converse; it is the greatest such relation and lies inside the core and inside the core of the converse net (`«位根»`, `«位核含于核»`, `«位核含于逆核»`). Ending is invariant under the core (`«亡于核下不变»`). Beginning is invariant under the core of the converse net (`«生于逆核下不变»`, stated since 1 November 2026), so under the two-way core (`«生于位核下不变»`), and abiding under the two-way core (`«位同则住同»`). The forward core can tell beginnings apart (`«生于核下可变之例»`), and the core of the converse net relates pairs the core does not (`«逆核不含于核之例»`). Altering is a relation between two occurrences with a finer class, not a mark of one occurrence, and is not read off the core (01 §4). The two-way core is the greatest bisimulation inside `E` (in general `E ∩ E˘`, as for the core) for `R` and its converse together: the standard notion of bisimulation (Blackburn, de Rijke and Venema 2001, §2.2) taken for the basic temporal language, with future and past, whose models carry a relation and its converse (their §1.3, Example 1.25). **Standing and use.** Root theorems (01 §4): "A position is fixed by all its relations, before as well as after." (ch. 1) **Reading and class.** None.

Known weaknesses.

- Two moments of a stationary process lie in one two-way core class and are separated only by carrier equality (L-02).

Questions.

1. Is that identification with temporal bisimulation correct?
2. (optional) Altering is read as a relation between two occurrences with a finer class. Is there a sense in which it should be read off the two-way core, and if so, under which hypothesis on the finer class?

L-09 The map at two grains: 延 coinductive, 尽 inductive

```
def «延» (R L : «际» α) (x : α) : Prop := ∃ S : α → Prop, S x ∧ ∀ z, S z → ∃ w, S w ∧ «续» R L z
w

inductive «尽» (R L : «际» α) : α → Prop
  | mk {x : α} : (∀ y, «续» R L x y → «尽» R L y) → «尽» R L x

theorem «延尽不并» {R L : «际» α} {x : α} (h : «尽» R L x) : ¬ «延» R L x

def «生成» (R L E : «际» α) (x : α) : Prop :=
  ∃ S : α → Prop, S x ∧ (∀ z, S z → ∃ w, S w ∧ «续» R L z w) ∧
    ∀ z, S z → ∃ u v, S u ∧ S v ∧ «所及» R L z u ∧ «传» R L E u v

theorem «生守相杂» : «生成» fork «—» «静» .root ∧ «守» fork «—» «静» .root

theorem yan_line_iff_dependentChoice :
    (∀ (α : Type) (R : «际» α) (x : α), «延» R «—» x → ∃ f : Nat → α, f 0 = x ∧ ∀ n, R (f n) (f
(n + 1))) ↔
      Onward.DependentChoice
```

Files: `Becoming/GrammarMap.lean` (`fork` and `«生守相杂»` in namespace `Map`), `Becoming/ProcessRepresentation.lean`. **Meaning.** 延 (glossed *goes on along some branch*) is coinductive: `x` lies in some predicate `S` each member of which has a continuation in `S`. It names no sequence of points. On relations on a type of the first universe, under the full class, obtaining one from every such point is exactly dependent choice (`yan_line_iff_dependentChoice`, 01 §8.1, stated on 29 October 2026), the shape `runs_iff_dependentChoice` calibrates for processes (L-18); within finitely many configurations a sequence comes with nothing chosen (`yan_line_of_list`). 尽 (glossed *ends on every branch*) is inductive accessibility: the least predicate holding at `x` wherever it holds at every continuation of `x`. The two exclude each other. That 尽 excludes an infinite chain of continuations from `x` follows in one step from «延尽不并» (the points of a chain form a family witnessing 延); the converse, from no infinite chain to 尽, depends on branching, decidability and bar induction or the fan theorem, and is not stated. Ending on every branch gives reaching an end (`«有终»`) only up to double negation (`«尽则非无终»`), with decided continuation an end (`«尽而续可定则有终»`), and for every model exactly under excluded middle (`«尽则有终当且仅当排中»`, L-11; 02). Generation is such an `S` containing `x` with, ahead of every member `z`, a carrying renewal between members `u`, `v`, `u` reached from `z`; keeping, such an `S` with no carrying renewal between members. Where each occurrence conditions at most one other, they exclude each other (`«单行则生守不并»`); on a branching relation they can share a point. The root's renewal without end (`«生生»`, the greatest family each member of which renews into a member) and ceasing (`«息»`, the inductive predicate holding where everything an occurrence renews into ceases) are the same two forms read along *renews into* (`«复生至»`) under the full class (`«生生即延»`, `«息即尽»`; 01 §6). **Standing and use.** Root theorems and calibrations; the regions generation, keeping, degeneration (02 §1). **Reading and class.** [AI reading, D65]: 生成 is continuation at two grains, not 生生.

Known weaknesses.

- Neither one-step corollary is stated in Lean (for 尽, no infinite chain of continuations, from «延尽不并»; for 息, no infinite chain of renewals, from «生生不息»); the converses depend on the hypotheses of Q23.

- Renewing and ceasing do not exclude each other: by «自不复生则息» at each occurrence it renews into, an occurrence that renews ceases where nothing renews again on the lines its renewals begin (a hand step, 01 §6). The chapter: "the definitions do not make renewing and ceasing exclude each other, though no model of both is given in Lean." (ch. 1) That what does not renew without end ceases is not proved (01 §6).

Questions.

1. Is 尽, inductive accessibility, the right constructive reading of *ends on every branch*, and under which hypotheses (finite or decided branching, decidable bars, bar induction, the fan theorem) should its relation to *every path is finite* be stated?
2. Is 延, a union of post-fixed predicates, the right coinductive reading where dependent choice is not assumed?
3. Are generation and keeping well defined as stated, and is the mixed region `«生守相杂»` the expected consequence?

L-10 The strong forms and coverage

```
def «全生成» (R L E : «际» α) (x : α) : Prop :=
  «不穷» R L x ∧ ∀ y, «所及» R L x y → ∃ z w, «所及» R L y z ∧ «传» R L E z w

def «全守» (R L E : «际» α) (x : α) : Prop :=
  «不穷» R L x ∧ ∃ y, «所及» R L x y ∧ ∀ z w, «所及» R L y z → ¬ «传» R L E z w

def «有终» (R L : «际» α) (x : α) : Prop := ∃ y, «所及» R L x y ∧ «亡» R L y

theorem «全三域之周» (em : ∀ P : Prop, P ∨ ¬ P) (R L E : «际» α) (x : α) :
    «全生成» R L E x ∨ «全守» R L E x ∨ «有终» R L x
```

File: `Becoming/GrammarMap.lean`. **Meaning.** Three forms read the whole coarse line; they exclude each other pairwise, and with excluded middle every point lies in one. **Standing and use.** Root theorems (02 §1). **Reading and class.** None.

Known weaknesses.

- `«全三域之周»` takes excluded middle for all propositions, and that every point of every model lies in one of the three is exactly excluded middle (`«全三域之定当且仅当排中»`, L-11); whether local decidedness suffices for a given model is not stated.
- The some-branch regions do not cover, and the chapter's counterexample is now in the Lean. The two keepings differ: "守 forbids renewal throughout its family where 全守 allows an earlier one" (ch. 2). So: "The line that renews once and then stays is, at its start, none of 生成, 守 and 退化, and is 后守 and 全守." (ch. 2) (`Map.Once.not_generates`, `Map.Once.not_keeps`, `Map.Once.not_degenerates`, `Map.Once.keepsAfter`, `Map.Once.keeping`). Keeping from some point on (`«后守»`) closes the gap: with excluded middle every point is 生成, 后守 or 退化 (`«三域之周»`), and that every point of every model is one of them is exactly excluded middle (`«三域之定当且仅当排中»`).

Questions.

1. (optional) Would local decidedness (of having a successor and of carrying renewal at each reached point) suffice for `«全三域之周»`, and is that its exact strength?

2. Is keeping from some point on (`«后守»`) the right fourth region for the gap `Map.Once` shows, and is its name exact?
3. Do the names generation, keeping and degeneration fit regions that may not cover every point?

L-11 Excluded-middle calibrations of the marks and the map

```
theorem «四相之定当且仅当排中» :
    (∀ (α : Type) (R E : «际» α) (x : α), «生» R E x ∨ «有前» R E x) ↔ ∀ P : Prop, P ∨ ¬ P

theorem «延尽之定当且仅当排中» :
    (∀ (β : Type) (R L : «际» β) (x : β), «延» R L x ∨ «尽» R L x) ↔ ∀ P : Prop, P ∨ ¬ P

theorem «不穷有终之定当且仅当排中» :
    (∀ (β : Type) (R L : «际» β) (x : β), «不穷» R L x ∨ «有终» R L x) ↔ ∀ P : Prop, P ∨ ¬ P

theorem «全三域之定当且仅当排中» :
    (∀ (β : Type) (R L E : «际» β) (x : β), «全生成» R L E x ∨ «全守» R L E x ∨ «有终» R L x) ↔
      ∀ P : Prop, P ∨ ¬ P

theorem law_iff_em :
    (∀ w : World, w.Finite → (w.ArisingRecurs ↔ w.CeasingRecurs)) ↔ ∀ Q : Prop, Q ∨ ¬ Q

theorem «尽则有终当且仅当排中» :
    (∀ (β : Type) (R L : «际» β) (x : β), «尽» R L x → «有终» R L x) ↔ ∀ P : Prop, P ∨ ¬ P

theorem generates_iff_changes (b : Nat → Bool) :
    «生成» step «—» (fun n m => b n = b m) 0 ↔ ∀ y, ∃ n, y ≤ n ∧ b n ≠ b (n + 1)

theorem keepsAfter_iff_settles (b : Nat → Bool) :
    «后守» step «—» (fun n m => b n = b m) 0 ↔ ∃ z, ∀ n, z ≤ n → b n = b (n + 1)

theorem «生成后守之定则LPO»
    (decides : ∀ b : Nat → Bool,
      «生成» step «—» (fun n m => b n = b m) 0 ∨ «后守» step «—» (fun n m => b n = b m) 0)
    (p : Nat → Bool) : (∃ n, p n = true) ∨ ∀ n, p n = false
```

Files: `Becoming/Grammar.lean`, `Becoming/GrammarMap.lean`, `Becoming/Space.lean`, `Becoming/MapRegions.lean` (the last three theorems, in namespace `Map`). **Meaning.** Deciding, in every model, whether an occurrence begins or has a predecessor; whether a point goes on or ends on every branch; whether a coarse line is without end or reaches an end; and in which of the three strong forms a point lies: each is exactly excluded middle. So is the arising–ceasing law asked of every finite world without decided activity (in a finite world arising recurs exactly when ceasing does; `World.law_iff_em`), which 01 §8.2 and 08 §5 rate a lemma of that law; 01 §8.2 also proves the law exactly equivalent, both being exactly excluded middle, to the first of the four, deciding the marks over every carrier and relation (`World.law_iff_grammar_positions`, in `Becoming/Space.lean`). So is a line that ends on every branch reaching an end, for every model (`«尽则有终当且仅当排中»`). The last three theorems fix one process, the successor on the numbers under the full class, and vary a sequence of truth values read as a class: its first step is 生成 exactly where the sequence changes at arbitrarily late steps, and 后守 exactly where it is constant from some step on. Placing every sequence in one of the two gives LPO; for a sequence that turns until a given one first holds, deciding 后守 gives LPO for that one (`Map.«后守之定则LPO»`) and deciding 生成 only WLPO (`Map.«生成之定则WLPO»`). **Standing and use.** Calibrations as 01 and 02 label them (01 §3,

02 §1), the World law a lemma. What 02 says the need means: "The boundary between going on and ending is where a line's going on is not decided" (ch. 2). **Reading and class.** None.

Known weaknesses.

- Each proof of the first four encodes an arbitrary proposition `P` as the relation `fun _ _ => P` on `Unit`; `«尽则有终当且仅当排中»` instead gates a one-step relation on `Bool` by `P`, so `P` still sits in the relation. For the World's law the method chapter rates the same pattern a lemma: "the need comes from activity being an arbitrary proposition, not from change" (ch. 8). 06 §四 already rates two similar results on the classical readings lemmas, not calibrations, since each concerns only the logic of the representation, not change: 相待之还 (the return of correlatives), exactly double-negation elimination; 必或不当 ("of two, one does not fit"), exactly weak excluded middle.
- [this dossier, AI] By the same rule the grammar's four may be lemmas too. The same decision is called a calibration at the marks (01 §3, and 08 §5's excluded-middle row, "deciding the four marks on every line") and a lemma at the World law, which the pure core proves equivalent to it.
- Since 30 October 2026 the chapter separates these from a need on one line: "On a single line the need is not of the logic alone." (ch. 2) [this dossier, AI] `Map.«后守之定则LPO»`, `Map.«生成之定则WLPO»` and `Map.«生成后守之定则LPO»` reduce a Boolean sequence, as the Markov calibration of L-17 does, and encode no arbitrary proposition, so the first weakness above does not reach them. As stated each gives its principle one way; the proofs show the equivalence at each coded sequence, which no theorem states. By the two equivalences above, the placement is the decision of a Σ^0_2 statement, and LPO is the part derived: "is itself the stronger decision whether each changes without end or settles" (ch. 2). The module header adds that without countable choice the core does not give the placement back from LPO; that is not proved in Lean.

Questions.

1. By the theory's own criterion (a calibration says something about the unfolding), are these calibrations or lemmas?
2. (optional) Are local forms (decidedness of having a predecessor at a given point) the exact strength the marks and regions need?

Part 2. Testing and the Hennessy–Milner converse

L-12 Testing by stages

```
def «层» (R E : «际» α) : Nat → «际» α
  | 0 => E
  | k + 1 => fun x y => «层» R E k x y ∧
      (∀ v, R x v → ∃ v', R y v' ∧ «层» R E k v v') ∧
      (∀ v', R y v' → ∃ v, R x v ∧ «层» R E k v v')

theorem «可分则不在核» {R E : «际» α} {x y : α} (h : ∃ k, ¬ «层» R E k x y) : ¬ «核» R E x y

theorem «恒温与平衡» :
    «核» Room.closed Room.alike Room.balanced Room.held ∧
      ¬ «核» Room.opened Room.alike Room.balanced Room.held
```

File: `Becoming/GrammarTesting.lean`. **Meaning.** Finite stages approximate the core from above: stage 0 is the class, stage `k + 1` matches steps both ways at stage `k`; the core lies in every stage (`«核在每层之内»`), so what a finite stage separates is not in the core. The stages are the finite-depth approximants of bisimilarity (Milner's `~k`; Hennessy and Milner 1985; Milner 1989), started from `E`; this direction needs no image-finiteness. A four-element witness: two warm rooms are in one core class of *warm or not* under one relation and separated at stage 1 under a larger relation that includes the window's opening. The converse is L-13 and L-14. **Standing and use.** Root theorem and witness: testing within the net (03 §3). **Reading and class.** 《荀子·正名》's 验 (testing) is cited in the module; the reading is the classical dossier's.

Known weaknesses.

- [this dossier, AI] The witness is two relations on four elements, the second containing the first, under which one class has different cores; it shows that a larger relation can have a finer core. Reading the second relation as the first with the window's opening added is the model's. Since 1 November 2026 this is less bare: at a margin of one draw the two relations, from the two rooms compared, are the steps of one rule of rooms, with the window's opening in its net (`Rooms.toy_branches`, `Rooms.toy_verdict`, in `Becoming/RoomNet.lean`), and at a margin of `n + 1` draws *warm or not* first tells the unheated room from any heated one at stage `n + 1`, while two heated rooms stay in one core class (03 §3). Stipulating the branches is removed; that a net read so is a test is still the model's.

Questions.

1. Is that credit accurate for stages started from a class?
2. Does the module's claim, "A wider net tells apart what a narrower one does not" (the GrammarTesting module), say more than that two relations have different bisimilarities?

L-13 The converse in the grammar: refuted, and proved under a premise

```
def «诘» (R E : «际» α) : Nat → «际» α
  | 0 => E
  | k + 1 => fun u u' => E u u' ∧ (∀ v, R u v → ∃ v', R u' v' ∧ «诘» R E k v v') ∧
      (∀ v', R u' v' → ∃ v, R u v ∧ «诘» R E k v v')

theorem «核则不可致诘» {R E : «际» α} {u u' : α} (h : «核» R E u u') (k : Nat) : «诘» R E k u u'

theorem «不可致诘而有常则混而为一» {R E : «际» α} (hE : «则» E («逆» E))
    (hc : «有常» R (fun u u' => ∀ k, «诘» R E k u u')) {u u' : α} (h : ∀ k, «诘» R E k u u') :
    «核» R E u u'

theorem «致诘不尽» :
    ∃ (β : Type) (R : «际» β) (u u' : β), (∀ k, «诘» R «—» k u u') ∧ ¬ «核» R «—» u u'
```

File: `Becoming/GrammarCandidates.lean`. **Meaning.** The core is separated at no stage of inquiry. The converse fails unconditionally: in an infinitely branching net (one node to every finite chain, another also to an endless one) two nodes agree at every finite stage and are not related by the core (no one class of the core holds both); this is the standard image-infinite example (Hennessy and Milner 1985; Milner 1989). It holds where the class is symmetric and the relation *separated at no stage* is itself constant, by the usual coinduction step (that relation is put into the core's definition). Classically, image-finiteness makes the limit relation constant: the convergence of the approximants, which is distinct from the modal characterization by Hennessy–Milner logic. **Standing and use.** Root theorems and a witness (03 §3). **Reading and class.** "此三者，不可致詰，故混而為一。" (LZ-14-002) (*these three cannot be inquired into to the end, and so they merge into one*; 老子 14) classified 乙 and proved under a premise: 致詰 is read as inquiry by finite stages. The reading is the classical dossier's.

Known weaknesses.

- Two stage notions coexist: `«诘»` asks `E` at each stage, `«层»` the previous stage; "which no theorem yet relates to the stages 层" (ch. 3).
- [this dossier, AI] The premise (the limit relation is constant) is global and the conclusion concerns one pair; no theorem derives the premise from an independently stated condition such as image-finiteness with decided steps.
- [this dossier, AI] Neither the module nor the chapter credits the countermodel.
- [this dossier, AI] 03 §3 describes the two nodes loosely: "lie in no class of the core" (ch. 3). The core of a class is itself a class (`«核为类»`), so each node lies in its own core class; what is proved is that no one class of the core holds both.

Questions.

1. (optional) Which independently stated conditions ensure that the limit relation is constant, and can one be proved in the pure core?
2. Is the countermodel described correctly?
3. Should `«诘»` and `«层»` be merged, or proved to agree?

L-14 The Hennessy–Milner limit and LLPO

A script outside the Lean package, not part of its checked build, `_process/research/2026-10-05-generative-foundation-review/root-foundation/development/one-unfolding-root/review-checks/HM.lean`:

```
def Within (U : Unfolding) (E : U.Way → U.Way → Prop) (u u' : U.Way) : Prop :=
  ∃ R : U.Way → U.Way → Prop, (∀ x y, R x y → E x y) ∧ (∀ x y, R x y → R y x) ∧ Carries U R ∧ R
u u'

theorem limit_gives_llpo
    (limit : ∀ α, (∀ k, Stage (U α) top k .u .u') → Within (U α) top .u .u') : LLPO
```

Pure core, `Becoming/Likeness.lean`:

```
def LLPOForDecided : Prop :=
  ∀ γ : Nat → Prop, (∀ n, Decided (γ n)) → (∀ m n, γ m → γ n → m = n) →
    (∀ n, ¬ γ (2 * n)) ∨ ∀ n, ¬ γ (2 * n + 1)

theorem sameBranchingTests_fix_iff_llpoForDecided :
    (∀ (S : Type) (F G : Frame S Unit Empty) (s s' : S), OutcomesListed F → OutcomesListed G →
        SameBranchingTests F G s s' → Alike F G s s') ↔ LLPOForDecided
```

Meaning. The script builds, for each Boolean sequence with at most one `true`, a finitely branching unfolding in which two nodes agree at every finite stage; the limit form of the converse, asked of all of them, gives LLPO. In the pure core, on processes with one move and no claims whose outcomes are listed, *same branching tests give likeness* is exactly `EitherThroughout`, which is exactly LLPO for decided families. Beside it, on listed configurations with one move and no claims and with steps not decided, *same branching tests give likeness* is exactly excluded middle (`alike_of_sameBranchingTests_iff_em`), proved by encoding an arbitrary proposition as a frame, the pattern of L-11. With countable choice, LLPO is equivalent to weak König's lemma, a known result of constructive reverse mathematics that the Likeness module credits to Ishihara (1990) and Bridges and Richman (1987); the pure core assumes no choice principle, so comparisons of strength depend on the base theory. **Standing and use.** The script's direction is listed as open in 03 until a proof inside the package exists (03 §3); the pure core's result is a calibration (03 §3). **Reading and class.** None; LZ-14-002 as in L-13.

Known weaknesses.

- The script lies outside the package and has no record. Run again for this version with Lean 4.29.1 on a copy (the two `#print axioms` lines at its end), both theorems depend on propext and Quot.sound, from tactics, not on Classical.choice. It shows one direction only.
- [this dossier, AI] The script uses its own copies of the notions (an unfolding, a one-way carrying condition, and stages that ask the class at every stage, shaped like `«诘»` and unlike `«层»`), not the grammar's `«际»`, `«核»` or `«诘»`. No theorem relates them to the grammar, so the script is not a statement in the grammar.
- No theorem connects the processes' branching tests to the grammar's stages (L-07), so the pure core's exact calibration does not reach the grammar.
- 03 says of the principle on processes "whether that principle is weaker is not shown" (ch. 3). That stands for `EitherThroughout` against excluded middle; the Likeness module says showing it would

take a model outside the pure core. 03 omits the principle's name: the pure core proves it exactly LLPO for decided families (`eitherThroughout_iff_llpoForDecided`, `sameBranchingTests_fix_iff_llpoForDecided`), which uses Prop-valued decidedness; the pure core derives standard LLPO from it, and the converse only where decisions are data (`LLPO.decidable`).

Questions.

1. Is the calibration stated at its exact strength?
2. (optional) Does the converse direction hold: LLPO giving the limit form for finitely branching decided unfoldings?
3. (optional) Is LLPO for Prop-decided families equivalent to LLPO without a choice principle?
4. How do these calibrations relate to the constructive literature on Hennessy–Milner-type theorems, LLPO and weak König's lemma?
5. (optional) Does `EitherThroughout` give `NowhereDecided` or excluded middle, and does a principle weaker than excluded middle suffice where the moves or the claims are not listed? Both are recorded as not derived in `Becoming/Likeness.lean`.
6. By the criterion of L-11, is `alike_of_sameBranchingTests_iff_em` a calibration or a lemma?

Part 3. Omniscience and choice

L-15 Settling is LPO

```
def LPO : Prop :=
  ∀ s : Nat → Bool, (∃ n, s n = true) ∨ ∀ n, s n = false

theorem lpo_iff_settles :
    LPO ↔ ∀ (S : Type) [DecidableEq S] (l : List S), (∀ x, x ∈ l) → ∀ (A : Type) (P : Process S
A) (s : S),
      (¬ ∃ x a y, Reach P s x ∧ P.step x a y ∧ y ≠ x ∧ Reach P y x) →
        ∀ r : Run P, r.state 0 = s → Enough.Settles r

theorem lpo_of_settles (settle : ∀ r : Run process, r.state 0 = false → Enough.Settles r) : LPO
```

Files: `Becoming/Change.lean` (the definition of `LPO`), `Becoming/LastChange.lean`, `Becoming/Settling.lean`, `Becoming/HistoryIdeals.lean` (the second theorem in namespace Settling.OneWay; `Stretches.cut_decided_iff_lpo` in HistoryIdeals). **Meaning.** Over every finite world of decidable configurations, every run from a configuration with no reachable change that can be undone settles exactly under LPO; in a two-configuration one-way process, every run settling gives LPO; deciding whether a cut history has a latest stretch or goes on is LPO. **Standing and use.** Calibrations (08 §5): "What the need says: settling is deciding whether the run changes again." (ch. 8) The chapter cites Bishop (1967), Bridges and Richman (1987) and Ishihara (no work named) as the known source. **Reading and class.** None.

Known weaknesses.

- Worlds that no list holds are outside these laws. That WLPO does not give LPO is cited (Akama, Berardi, Hayashi and Kohlenbach 2004, in 08 §5) and not checked.

Questions.

1. Is the equivalence stated at its exact strength, and is the credit adequate?
2. Is that gloss an accurate description of what the need says?

L-16 Tails: WLPO and LPO as rules

```
def ChangedByNoCondition (f : (Nat → Nat) → Nat) : Prop :=
  ∀ (c : Nat) (α β : Nat → Nat), (∀ m, m ≠ c → α m = β m) → f α = f β

theorem changedByNoCondition_iff_rule :
    (∃ f : (Nat → Nat) → Nat, ChangedByNoCondition f ∧ ∃ α β, f α ≠ f β) ↔
      Nonempty (∀ s : Nat → Bool, Decidable (∀ n, s n = false))

theorem vanishDecision_iff_lpoRule :
    Nonempty (∀ α : Nat → Nat, Decidable (VanishFrom α)) ↔
      Nonempty (∀ s : Nat → Bool, Decidable (∃ n, s n = true))
```

File: `Becoming/Tail.lean`. **Meaning.** A number read off a sequence of levels that no single coordinate changes, yet that differs between two sequences, exists exactly where a rule decides of every Boolean sequence whether it is false throughout; deciding eventual vanishing is exactly the LPO rule. **Standing and use.** Calibrations, read as what a law of the whole and no part takes; used for the least coupling (08 §5). **Reading and class.** None.

Known weaknesses.

- The statements concern uniform rule forms, `Nonempty` of a decision function, which imply the propositions WLPO and LPO (`WLPO.of_rule`, `LPO.of_rule`); the reverse implications are not established, and no separation is proved (comparisons of strength with the constructive literature also depend on the base theory, L-14): "That the proposition WLPO gives a rule is not shown, since that step chooses a function from a relation." (ch. 8)

Questions.

1. Is calling these WLPO and LPO accurate, or should they carry the names of the rule forms only?
2. Is the first equivalence a form of a known link between discontinuous functionals and WLPO, and should it be credited?

L-17 Refutation, verification, double negation and Markov

```
theorem refutable_closed_iff :
    (∀ (S A : Type) (P : Process S A) (C : Ideals.Ideal (stretchOrder P) → Prop),
      Ideals.Refutable C → Ideals.ScottClosed C) ↔ ∀ φ : Prop, ¬ ¬ φ → φ

theorem refutable_passes_iff_markov :
    (∀ (S A : Type) (P : Process S A) (fails : Nat → Run P → Bool),
      ByPrefixes fails → Refutable (Passes fails)) ↔ Markov
```

Files: `Becoming/HistoryIdeals.lean` (namespace Stretches; beside it `Stretches.closed_refutable_iff` and `refutable_holds_iff`), `Becoming/Horizon.lean`, `Becoming/ModelStatements.lean`. **Meaning.** Histories are ideals of finite stretches. Asked of every process: refutable classes are closed, closed classes refutable, and a refutable reading holds at every adhering run, each exactly under double-negation elimination; a reading made of tests is refutable exactly under Markov's principle. **Standing and use.** Calibrations; the formal side of falsifiability (03 §5), with the safety–liveness decomposition (Alpern and Schneider 1985) credited, constructively. Since 30 October 2026 verifiability is also read on the grammar's terms, on one carrier of points and stretches: a reading is verifiable exactly where every point where it holds belongs to a stretch all of whose members it holds of (`Becoming.ModelStatements.verifiableBy_reading`). **Reading and class.** [AI reading, D26]: refutability is not made the criterion of scientific standing.

Known weaknesses.

- [this dossier, AI] Each is asked of every process. The double-negation calibrations fix the process (the process of `()`s, or of Boolean sequences) and encode an arbitrary proposition `φ` in what is asked about: the class of histories or the reading (`… ∨ φ`), or a history extended where `φ` holds. So the L-11 question applies to them, though there the proposition is the relation itself (`fun _ _ => P`); the Markov calibration instead reduces a Boolean sequence (the test at `n` fails where its move is `true`).

Questions.

1. Are these known in constructive topology or formal topology (closed sublocales, the double-negation nucleus), and how should they be credited?
2. Are calibrations universal over all processes informative, or would a calibration at a fixed process be the exact strength?
3. (optional) Do the two diagonal limit laws, `Limit.Free.settles_not_refuted` and `Limit.Sequences.recurs_not_verified` (no method fixed by the record refutes settling, or verifies recurrence, in the limit), need Markov's principle, under which each is proved? 03 §5 records this as open: "whether the two diagonal laws need Markov's principle is open" (ch. 3). Separately, two other laws each imply LPO, one way only (`Limit.Sequences.eventuallyFalse_lpo`, `Limit.Sequences.either_recurs_lpo`).

L-18 Runs, dependent choice and finite branching

```
def DependentChoice : Prop :=
  ∀ (α : Type) (R : α → α → Prop), (∀ x, ∃ y, R x y) →
    ∀ x, ∃ f : Nat → α, f 0 = x ∧ ∀ n, R (f n) (f (n + 1))

theorem runs_iff_dependentChoice :
    (∀ (S A : Type) (P : Process S A) (K : S → Prop), KeepsMoving P K →
      ∀ s, K s → ∃ r : Run P, r.state 0 = s ∧ ∀ n, K (r.state n)) ↔ DependentChoice

theorem step_iff_either :
    (∀ (S A : Type) (P : Process S A), FinitelyBranching P → ∀ s, Unbounded P s →
      ∃ t, (∃ a, P.step s a t) ∧ Unbounded P t) ↔ EitherThroughout
```

Files: `Becoming/Onward.lean`, `Becoming/Lasting.lean` (namespaces `Onward`, `Lasting`). **Meaning.** A run from every member of every family that keeps moving is exactly dependent choice. In a finitely branching

process an unbounded configuration having an unbounded successor is exactly `EitherThroughout` (L-14). `Lasting.lifting` separates three steps: with that principle and finite branching an unbounded successor exists (`Unbounded.step_of_either`); a supplied function `choose` turns that existence proof into a successor with its move; `runOf` iterates the selection. So every unbounded configuration starts a run along unbounded ones, explicit relative to these data; no selector is obtained unconditionally. The same dependent choice reads the grammar's 延 (L-09). **Standing and use.** Calibrations (08 §5); the line from an occurrence never ending (`«不穷»`) yields an endless chain only with dependent choice (01 §2). **Reading and class.** None.

Known weaknesses.

- `choose` is data-level choice; `lifting` gives a run's existence, not an effective selection (the chapter says so). Weak König's lemma is credited in the Likeness and Lasting docstrings, not in the chapter.

Questions.

1. (optional) Is the hypothesis set of `lifting` minimal? `choose` acts only on a proof that an unbounded successor exists, which `EitherThroughout` with finite branching supplies; can that proof be had with less?
2. Should the relation of `step_iff_either` to König's lemma and weak König's lemma be stated in the chapter?

Part 4. The method

L-19 The pure-core discipline

The checker's assertion on every source declaration, `check_becoming.py`:

```
assert set(axioms) <= {"propext", "Quot.sound", "Classical.choice"}, (full, axioms)
assert name not in SPINE + PURE or not axioms, (full, axioms)
```

`SPINE` and `PURE` are the checker's lists of module names: the three foundation modules, the pure core and its instance and application ring. **Meaning.** In the chapter's words, "the pure core imports only the foundation and itself, and compiles from Lean's prelude (`Init`) alone, with the foundation's `import Std` removed" (ch. 8); every source declaration in the pure core and its foundation has an empty axiom report (no propext, Quot.sound or Classical.choice); the only constants with an axiom are generated injectivity lemmas, on propext, which no source declaration uses. **Standing and use.** A discipline the checker asserts on every run (08 §3). "An empty report shows that no logical axiom is used. It says nothing about whether the modelling fits an actual unfolding" (ch. 8). **Reading and class.** None.

Known weaknesses.

- [this dossier, AI] An empty report does not record which kernel features a proof uses: impredicative `Prop` (L-06), definitional proof irrelevance, large elimination, the primitive quotient, kernel computation by `decide`.

- The chapter gives two reasons for excluding propext and none for quotient soundness, or for function extensionality, which follows from it. First, from the relational start: "says that a proposition is nothing beyond them. Whether to assume it is therefore a question about lumping." (ch. 8) Second, transport along propext does not reduce, which defeats checks by computation (L-20). The heading "Stricter than constructive, and about logic only." (ch. 8) means that the report concerns logic and not the modelling, not that the exclusion has a logical motive.
- [this dossier, AI] Propositional extensionality and quotient soundness are constructively acceptable, so the extra strictness appears to concern computation (transport that reduces) rather than logic.
- Some cited results are outside it (the HM script, L-14; periphery modules with Mathlib).

Questions.

1. Is the discipline meaningful as stated? Is the strictness beyond constructivity about computation, and should the chapter give a reason for excluding quotient soundness and function extensionality?
2. Should the report also list impredicativity and other kernel features a declaration depends on?

L-20 propext as the lumping of every reading

```
def Extensional : Prop := ∀ a b : Prop, (a ↔ b) → a = b

theorem extensional_iff_lumped :
    Extensional ↔ ∀ g : Prop → Prop, Learning.Lumped relation (fun i => g (endOf i))

theorem byName_lumped_iff : Learning.Lumped relation (fun i => byName (endOf i)) ↔ Extensional
```

File: `Becoming/PropositionIdentity.lean` (namespace `PropositionIdentity`). **Meaning.** propext, stated as a proposition, holds exactly where every map from propositions to propositions is constant on interderivable propositions (*lumped*: a reading lumped by a relation gives alike values to related arguments), and exactly where the reading by name is. **Standing and use.** Theorem (any type); the reason the pure core states `↔` where `=` would need propext (08 §3). **Reading and class.** The case for excluding propext is argued in the chapter, not registered as a reading.

Known weaknesses.

- The claim that transport along propext does not reduce is "a kernel observation with Lean 4.29.1 on one value, not a theorem." (ch. 8)
- Function extensionality and quotient soundness are not treated.

Questions.

1. Is the equivalence anything beyond the folklore that propext is congruence of every predicate on `Prop` under `↔`, and does the word *lumping* add content?
2. Is the argument from loss of computation a sound reason to exclude propext from a core whose claims are propositions?

L-21 Expressible is not derived; the World as a model

```
theorem arises_iff_sheng {t : Nat} {f : F} :
    W.Arises t f ↔ W.active (t + 1) f ∧ «生» Steps.along W.activityClass (t + 1, f)

theorem carries_iff {x y : Nat × F} (start : W.active x.1 x.2) :
    «传» (continuationStep R) W.activityClass W.carrierClass x y ↔
      continuationStep R x y ∧ W.active y.1 y.2 ∧ ¬ (W.active x.1 y.2 ∧ R x.1 y.2 y.2)
```

Files: `Becoming/Arising.lean`, `Becoming/MapInstances.lean` (namespace `Web`), `Becoming/GrammarLaws.lean`. **Meaning.** Instance theorems state the World model's continuation family in the grammar: a formation arises exactly where the occurrence after the step is active and begins under the class of activity; from an active occurrence, a carrying renewal on the World is exactly a continuation step into an active occurrence whose formation was not both active at the earlier step and related to itself by `R` there. A law proved on a model passes to the grammar only by a dependency audit, a lifting theorem and a representation theorem. What has passed so far: since 26 October 2026 five shared laws (closure, balance as represented and not derived, finite runs, rest, ends on lines) hold for every carrier and relation, each anchor read at its model's relation both ways, closure for instance being `«所闭当且仅当同一或及»` (01 §7). Of the 117 distinct central declarations, on 2 November 2026 eleven are derived as cases of grammar laws through bridges checked in Lean, one is a case under a stated hypothesis, six are counting below the grammar and 99 stand at a named distance with a reason; none is a premise (08 §7). **Standing and use.** Theorems (World) and a method (08 §6, 01 §8.6). The audit: "Of these, 9,370 read no World field; 348 read only" (ch. 8). **Reading and class.** None.

Known weaknesses.

- Processes are represented both ways by their moves' relations (01 §8.1) and the World both ways for its relations, failing on its amounts (01 §8.6); most modules do not yet state their laws on the grammar (01, Open): "Until a law is lifted, it is stated with its model." (ch. 8)
- The sentences of 08 §6 on pairing, power and three-variable logic are cited from the literature and not checked there (L-01).

Questions.

1. Is audit, lifting and representation a sound criterion for a law of the grammar, and what form should the representation theorem take (a faithful functor, an equivalence of categories of models, an interpretation)?
2. In what sense is the grammar the theory's root while most laws are not lifted, and what would have to be lifted for the name to fit?
3. (optional) Can mutual interpretation with a named fragment of Lean (each compiled into the other, provability kept both ways), which 08 §6 leaves open, be shown, and which fragment is natural?

L-22 The World's premises read off one exchange

```
theorem withRest_balanced : w.withRest.Balanced

theorem listed_iff_lpo :
    (∀ L : Flow Nat Nat, (∀ t, Distinct (L.present t)) →
        ∀ t, ∃ l, Distinct l ∧ ∀ f, L.Exchanging t f ↔ f ∈ l) ↔ LPO

theorem respondsBelow_weakest {R : (Nat → C → Nat) → Nat → F → C → Nat → Prop}
    (sound : rel.Supports present R) (local_ : Local R) {l : Nat → C → Nat} {t : Nat} {f : F}
    {c : C} {n : Nat} (reads : R l t f c n) : rel.RespondsBelow present l t f c n
```

Files: `Becoming/Reading.lean` (namespace `World`), `Becoming/ExchangePremises.lean` (namespace `Flow`), `Becoming/OverExchange.lean` (namespace `ExchangeRelation`). **Meaning.** With the rest of the whole counted as one more relatum, every world balances; over every flow on `Nat` whose relata present at each step are listed without repetition, listing without repetition what exchanges at every step is exactly LPO; necessary response is the weakest sound reading of requirement that does not read the current level. **Standing and use.** Theorems (World) and a calibration: four premises of the World hold by the reading (01 §8.3). **Reading and class.** [AI reading, D65]: locality, and what internal relations mean here.

Known weaknesses.

- Counting the difference as a relatum makes balance hold by construction; the chapter grants it, "Balance is not removed: nothing comes from nothing becomes feasibility, a premise located in the law." (ch. 1)

Questions.

1. Is holding by the reading a derivation, or a definitional extension that relocates the premise?
2. Is `Flow.listed_iff_lpo` stated at its exact strength, and does it say something about exchange rather than about listing?

L-23 What the dynamics do not entail

```
def ReadsOnlyDynamics {D : Type} (law : D → (Pos → Nat → F → Prop) → Prop) : Prop :=
  ∀ d sig sig', law d sig → law d sig'

theorem ReadsOnlyDynamics.not_entails {D : Type} {law : D → (Pos → Nat → F → Prop) → Prop}
    (reads : ReadsOnlyDynamics law) (entails : ∀ d sig, law d sig → Unrestricted sig)
    (p : Pos) (f : F) {d : D} {sig : Pos → Nat → F → Prop} : ¬ law d sig

theorem not_rival_unrestricted_iff_stable (p : Pos) (f : F) :
    (∀ sig : Pos → Nat → F → Prop, ¬ Rival sig → Unrestricted sig) ↔ ∀ q : Prop, ¬ ¬ q → q

theorem «续无外随网» :
    «续无外» LineOfDescent.Endless.step «一» (fun (_ : Unit) => ((0, false) :
LineOfDescent.Endless.Occ)) ∧
      ¬ «续无外» TwoLines.step «一» (fun (_ : Unit) => ((true, 0, false) : TwoLines.Occ))

theorem ReadsNoOrder.not_entails {D : Type} {law : D → (S → S → Prop) → Prop}
    (reads : ReadsNoOrder law) {frame : Frame S A C} {living : C → Prop} {d : D}
    (entails : ∀ better, law d better → Comparison frame living better)
    {s t : S} (extension : Extension frame living s t)
    {better : S → S → Prop} : ¬ law d better
```

Files: `Becoming/Significance.lean`, `Becoming/PremiseWitnesses.lean`. **Meaning.** A *significance* `sig` says, for a standpoint, a step and a formation, whether from that standpoint the formation's continuation then has significance; the continuation condition `Unrestricted sig` says that it always does. A statement whose truth does not change with the significance, if it entails the condition for every significance, holds of nothing (where a standpoint and a formation exist); over every significance, that excluding every rival (a significance holding exactly within a region with something outside it) gives the condition is exactly double-negation elimination. Here the significance is a free parameter. Read instead as lying within a center's closure along a relation, the condition has no such parameter: it is exactly the grammar's `«续无外»` (every occurrence reached on every center's line), with that relation as the net and the full class as the class (`ownClosure_unrestricted_iff_noOutside`, 07 §4.5). That root form varies with the net: the endless line of descent meets it and two lines with no condition between them fail it (`«续无外随网»`), so no law holding of every net settles it. C has a theorem of the same pattern: a statement reading no order that entails C at a datum, on a frame with an extension, fails there for every order (`ReadsNoOrder.not_entails`), which is immediate, since "better" is free (08 §7, since 1 November 2026). **Standing and use.** Theorem (any type) and calibration, with the root form's variation on supplied nets and the last theorem on frames: the ethics needs premises, the continuation condition for scope (whose losses count) and C (that widening what can form is better) for order, neither asserted (07 §1). "That every World law reads only the dynamics is a fact about the representation" (ch. 7). **Reading and class.** None.

Known weaknesses.

- The proof instantiates the significance as the empty one; the method chapter rates it near-definitional.
- [this dossier, AI] *Reading only the dynamics* is semantic invariance, not a syntactic condition on statements.
- The first two theorems here quantify over the free significance, which is not stated on the grammar: "The free significance of §1 is not lifted onto the root." (ch. 7) Only the own-closure reading above is on the root, where 07 proves one of the condition's consequences; three more are proved on a web's level paths.

Questions.

1. Is invariance under change of significance the right formalization of reading only the dynamics, against syntactic forms of Hume's law (Prior 1960; Schurz 1997), and should these be credited?
2. Is the double-negation calibration informative about evaluation, or a lemma of the logic?

L-24 Widening as what every positive weighting ranks

```
theorem widens_iff_every_positive {items : List X} {a b : X → Bool} :
    WidensOn items a b ↔ ∀ w, Positive items w → weighed items w b < weighed items w a

theorem widens_iff_every_of_class {items : List X} {a b : X → Bool}
    {K : ((X → Bool) → (X → Bool) → Prop) → Prop}
    (weightings : ∀ w, Positive items w → K (ranks items w))
    (increasing : ∀ R, K R → Increasing items R) :
    WidensOn items a b ↔ ∀ R, K R → R b a
```

Files: `Becoming/Weightings.lean`, `Becoming/PremiseWitnesses.lean` (namespace `Weighting`). **Meaning.** On a finite list of items with two-valued admission, one course widens another (admits every item the other admits, and one more) exactly when every weighting by positive natural numbers on the items weighs it higher. The same holds for any class `K` of comparisons (binary relations on courses, not assumed complete, transitive or irreflexive) containing every positive weighting's comparison and only relations ranking each widening above what it widens. This is a finite, natural-weight form of the idea of Sen's intersection quasi-ordering (Sen 1970), widening being Pareto dominance, proved directly. Since 30 October 2026 this is also read on the root, as the grammar's widening `«⌐»` of what one course admits over another (`Weighting.grammar_widens_iff_every_positive`), the weights being a second sort the grammar has none of. Since 1 November 2026, with equality of items decided, widening on a listing is generated by widening by one item (`Weighting.widensOn_iff_reach`), so an order transitive on courses is increasing in admission exactly when it ranks every widening by one item above what it widens (`Weighting.increasing_iff_by_one`). **Standing and use.** Theorem (any type): C, the premise that orders, is decomposed into the unanimity of standpoints whose comparisons increase in admission (07 §4.4); the chapter credits Sen's intersection approach and Pareto dominance. **Reading and class.** None.

Known weaknesses.

- "Monotonicity at the standpoint is C's premise relocated: weaker, still a premise." (ch. 7) Weights are natural numbers and the listing finite.

Questions.

1. Is the credit above accurate, and what does the class version state beyond monotonicity?
2. Do natural weights and a finite listing limit the result in a way the chapter should state?

L-25 Lawvere's diagonal as an unfolding

```
def Says (same : B → B → Prop) (ψ : A → A → B) (g : A → B) : Prop :=
  ∃ a, ∀ x, same (ψ a x) (g x)

def gap (f : B → B) (ψ : A → A → B) (x : A) : B := f (ψ x x)

def Moves (same : B → B → Prop) (f : B → B) : Prop := ∀ b, ¬ same b (f b)

theorem gap_unsaid {same : B → B → Prop} {f : B → B} (moves : Moves same f) (ψ : A → A → B) :
    ¬ Says same ψ (gap f ψ)

theorem Unfolds.never_returns (U : Unfolds same f φ) (moves : Moves same f) (n k : Nat) :
    ¬ ∀ g, Says same (φ (n + 1 + k)) g → Says same (φ n) g
```

File: `Becoming/Diagonal.lean` (namespace `Diagonal`). **Meaning.** For a map moving every value, no member of a family of readings indexed by what it reads says the diagonal gap, for any likeness `same`. For inhabited `A` and `B`, with decidable equality on `B` and likeness taken as equality, some family says every reading exactly when all values of `B` are equal (`Diagonal.complete_iff_alike`). A family that keeps what it said and next says its last gap never returns. **Standing and use.** Theorem (any type): how far a description made within what it describes can complete itself (03 §6): "Lawvere's fixed point as an unfolding; coding and provability are not stated." (ch. 3) **Reading and class.** None; the liar is read as a rhythm with no still value. That rhythm is the liar's revision sequence in the revision theory of truth

(Herzberger 1982; Gupta 1982; Gupta and Belnap 1993), here for one sentence, with no limit rules or truth predicate.

Known weaknesses.

- [this dossier, AI] The gap theorem is Cantor's diagonal in Lawvere's form (Lawvere 1969), which 03 credits; the unfolding iterates it. Neither the chapter nor the module credits revision theory for the rhythm.

Questions.

1. What does the unfolding (completion that never returns) state beyond iterating the diagonal argument?
2. How does the reading of the liar as a rhythm relate to the revision theory's treatment of it? The three-valued counterpart, where the liar rests undecided, is L-27.

L-26 The completion criterion K1–K7

No Lean statement: a tool, `tools/research/completion.py`, run whenever the results change. **Meaning.** Seven checks: coverage, unity of the base families, premises named with witnesses, representation over dense amounts, enclosure of supplied parameters, one ethics chain, an open register. **Standing and use.** A method (08 §12): "On the registered coverage, the configured checks have no finding. That is all they establish." (ch. 8) **Reading and class.** None.

Known weaknesses.

- "so patterns match source text, not elaborated terms: a premise named only through a definition the statement unfolds is not found." (completion.py) K4 and K5 look only at the patterns their coverage file lists; "So the criterion is completion in a qualified sense, not a proof that every premise is named, and not a proof of necessity." (ch. 8)

Questions.

1. Is a check on source text a meaningful criterion of completion, and what would a logician require instead (for example, an audit of hypotheses on elaborated terms)?
2. Is K3's treatment of logical premises (a witness meeting decidedness, a proof it is never refuted, a calibration) sound?

Part 5. Further results in logic

L-27 Revision with an undecided value

```
def stage (sys : Nat → Term) (n : Nat) : Nat → Value := (Law.run (revise sys) blank).state n

theorem least_fixed (closed : Closed sys N) :
    (∀ i, i < N → revise sys (stage sys N) i = stage sys N i) ∧
      ∀ w : Nat → Value, (∀ i, i < N → eval w (sys i) = w i) → ∀ i, i < N → Le (stage sys N i)
(w i)

theorem chain_exact (N : Nat) : Steady chain (N + 1) (N + 1) ∧ ¬ Steady chain (N + 1) N

theorem liar_rests : ∀ n i, stage liar n i = u

theorem undecided_of_moves (closed : Closed sys N)
    (moves : ∀ b : Nat → Bool, ∃ i, i < N ∧ eval₂ b (sys i) ≠ b i) {w : Nat → Value}
    (fixed : ∀ i, i < N → eval w (sys i) = w i) : ∃ i, i < N ∧ w i = u
```

File: `Becoming/Undecided.lean` (namespace `Undecided`). **Meaning.** With an undecided value below true and false and Kleene's strong three-valued tables, revising `N` sentences that speak only of one another, from all undecided, settles within `N` steps at the least fixed point: Kripke's construction (1975) for finitely many sentences, as the module credits, where each sentence rises at most once, so no limit stage is needed (the count `Undecided.settles_within`). The bound is exact (a chain of `N + 1` sentences is not steady at step `N`). Where two-valued revision has no fixed point, every three-valued fixed point leaves a sentence undecided, and the liar rests undecided. No map monotone in information moves every value (`Undecided.monotone_not_moves`). **Standing and use.** Theorem on the module's own syntax (terms over `Nat`-indexed sentences with the two constants, negation, conjunction and disjunction), for `N` sentences whose terms mention only sentences below `N` (`Closed`); no type parameter occurs, though 03 labels it "any type" (03 §6; record); with L-25 it bounds how far a description within what it describes completes itself (08 §6). **Reading and class.** "Undecided is what revision has not decided, not a third truth." (ch. 3) Not a corpus unit.

Known weaknesses.

- The chapter credits no source at the theorem. The module does: "The tables are Kleene's strong three-valued tables; settling at the least fixed point from all undecided is Kripke's construction (1975) restricted to finitely many sentences" (the Undecided module).
- Infinitely many sentences, where the least fixed point needs a limit stage, and fixed points other than the least are not derived (module header).

Questions.

1. Is the finite specialization stated correctly, and is the module's credit adequate for the chapter?
2. Is the exact bound (`Undecided.chain_exact`) recorded in the literature?
3. Taken with L-25, how does the reading of the liar on both sides (two values, a rhythm that never rests; three values, resting undecided) relate to revision theory and to Kripke's fixed point?

L-28 Curry and contraction (older modules)

```
theorem curry_fixed_point {C Q : Prop} (h : C ↔ (C → Q)) : Q

theorem admissible_contraction_iff_duplicate {P : Process S E} {join : S → S → S}
    (unit : S) (hu : ∀ x, join unit x = x) (hf : StepFrame P join) (a : S) :
    AdmissibleContraction P join a ↔ Reach P a (join a a)
```

Files: `experiments/integrated-correction/general-framework/generative-basis/lean/deeper/ReflectiveIdentity.lean` (namespace Generative.ReflectiveIdentity, an older module outside the Becoming package; beside it RecognitionalBinding.lean) and `experiments/integrated-correction/general-framework/generative-basis/lean/deeper/GeneratedContraction.lean`. **Meaning.** Admitting `C ↔ (C → Q)` under implication and reuse of a hypothesis yields any `Q`. In an object calculus of configurations with identity, steps, cut and framing but no contraction, given a left unit and steps compatible with each declared context, contraction is admissible at `a` exactly where duplication `a ⇝ a ⊗ a` is reachable. In that calculus derivability is reachability, and replacing `y` by `x` in every left context is exactly `x` reaching `y` (derivation_iff_reach and replacement_iff_reach, in the same file). **Standing and use.** Theorems (any type, with the criteria, the fixed-point equivalence and the term language supplied; any process), in older modules checked on their own, each with an empty axiom report (08 §6, contraction, self-reference); a companion module for contraction has some declarations on `propext` and `Quot.sound`. From the first and its companions 08 concludes "So self-reference does not certify itself, and the reference back of a bound use is stipulated rather than derived." (ch. 8) **Reading and class.** None.

Known weaknesses.

- 08 claims no novelty for the contraction theorem: "no novelty over linear, graded or Petri-net logics is claimed" (ch. 8). Binary duplication gives every finite multiplicity, not the exponential modality.
- [this dossier, AI] The Curry statement, which 08 credits to Curry, is the propositional form of Curry's paradox with the fixed point taken as a hypothesis; no self-referential sentence is constructed.

Questions.

1. Is the contraction theorem a known fact of substructural or resource logics (linear logic, bunched implications, Petri-net semantics), and how should it be credited?
2. Does 08's conclusion about self-reference follow from these statements?

L-29 The other calibrations of 08 §5

No Lean block: the statements are long; each is named and linked, and the third column gives its quantifiers and hypotheses. The middle column is 08 §5's account of what the need says.

Principle	What needing it says (08 §5)	What the Lean states	Lean
Decidedness, local	a count or a named step exists only where its object is decided	any list, any property: a count exists exactly where the property is decided at each member; no principle	`counts_iff_decided` (`Becoming/Decidedness.lean`; 08)
Weak excluded middle	where it is open whether one move betters another, no move need be known to be chosen	every type and relation: each nonempty list on whose members the relation is irreflexive and transitive has an undominated member, exactly where every proposition is weakly decided (`¬ p ∨ ¬ ¬ p`); the frame form, the same for a configuration's listed options	`exists_undominated_iff_weakly` (`Becoming/Decidedness.lean`), `FrontierFrames.choice_exists_iff_weakly` (`Becoming/FrontierFrames.lean`)
Double-negation elimination	reach without a path	every process, seed predicate and configuration: where no invariant property separates it from the seeds, it is reached from a seed (`∃ s, seed s ∧ Reach P s x`, a finite path), exactly under the principle	`fromSeed_of_not_separated_iff_dne` (`Becoming/SeedReach.lean`)
Excluded middle	names carry one decided identity more than relations	every World: indiscernibility of any two relata is decided; (second law) every list of relata has a number up to indiscernibility; each exactly under the principle. Carrier equality is not at issue	`World.indiscernible_decided_iff_em`, `World.numbers_indiscernible_iff_em` (`Becoming/Listing.lean`; 08)

Standing and use. Calibrations (08 §5). **Reading and class.** None.

Known weaknesses.

- [this dossier, AI] 08's gloss of the double-negation row, "reach without a path" (ch. 8), misdescribes the law: its conclusion is reach, which carries a path; what needs the principle is the step from the absence of an invariant separation to such a path.

Questions.

1. By the criterion of L-11, which of these are calibrations and which lemmas?

2. Is each stated at its exact strength, and is any a known equivalence of constructive reverse mathematics that should be credited?

The names

What the items use: how names are written, the classes of the quoted units, the grammar's vocabulary and the theorem names.

How names are written. Guillemets make a Chinese word one identifier: `«续» R E x y` reads "`y` continues `x` under `E`". A dotted name such as `Web.carries_iff` is a declaration in a namespace (`Web`); the line under each block names the file and, where needed, the namespace.

The classes of quoted units. Each sentence of the classical texts the theory reads (a *unit*, quoted with its id) was classified by an AI coder into one of eight classes (the re-coding codebook, §三); the three that occur here are 甲, a definition; 乙, a candidate theorem of the grammar, with its outcome in Lean (proved; proved under a premise; refuted); 己, a contrary reading, where the text rejects a thesis of the theory.

The grammar's vocabulary.

Name	Gloss	Name	Gloss
`«际»`	relation `α → α → Prop`	`«续»`	continuation `R ⊓ E`
`«而»`, `«逆»`	composition `R;S`, converse `Rˇ`	`«相续»`	line: reach of continuation
`«合»`, `«非»`	meet, complement (pointwise negation)	`«有前»`, `«有后»`	has a predecessor, a successor
`«则»`, `«谓»`	inclusion, pointwise equivalence	`«生»`, `«住»`, `«亡»`	begins, abides, ends
`«一»`, `«静»`	full relation ("the one"), identity	`«异»`	alters: continues under `E` while `F` tells apart
`«及»`	reach: transitive closure	`«有常»`	constancy `Eˇ;R ⊆ R;Eˇ`
`«无环»`	acyclic: nothing reaches itself	`«核»`, `«位核»`	the core of a class; the two-way core
`«俱有»`	co-presence: mutual reach	`«不穷»`	the line never ends
`«类»`	class: an equivalence relation	`«所及»`	reached on the coarse line (or the point itself)
`«延»`, `«尽»`	goes on along some branch (coinductive); ends on every branch (inductive)	`«传»`	carrying renewal: the coarse line continues into a fine beginning
`«生成»`, `«守»`	generation, keeping (along some branch)	`«全生成»`, `«全守»`	generation, keeping (the whole coarse line)

Name	Gloss	Name	Gloss
«有终»	reaches an end	«阶全生成»	generation at an order of a sequence of classes
«后守»	keeping from some point on	«定后», «同行»	the successor is fixed `R˘;R ⊆ 静`; going alike `R˘;E;R ⊆ E`
«所闭»	within a relation's closure	«广»	widens: holds the other and more
«复生至»	renews into	«生生», «息»	renewal without end (coinductive); ceasing (inductive)
«层»	stage `k` of the core	«诘»	stage `k` of inquiry (asks the class at every stage)
«续无外»	the continuation condition along the net (L-23)		

Theorem names, glossed word for word; a name of the form «X之定当且仅当排中» reads "deciding X is exactly excluded middle".

Item	Name	Gloss
L-02	«缘先果后当且仅当无环», «无环则及为序»	condition before, result after, exactly where acyclic; acyclic, so reach is an order
L-02	«相由则俱有», «俱有为一念»	mutual conditions are co-present; co-presence is one cluster (literally "one thought-moment")
L-03	«任一缘可为续», «粗化», «蜡烛之合», «分叉»	any condition can be a continuation; coarsening; the candles' merge; fission
L-04	«四相无自立», «两端分则中有标»	no mark stands by itself; ends told apart, so a marked step between
L-04	«亡于粗则亡于细», «生于细而续于粗则异»	ending under the coarser, ending under the finer; begun under the finer and continued under the coarser, an alteration
L-05	«有常则续不尽», «有亡之续则不有常»	constancy, so continuation never runs out; a continuation with an end, so not constant
L-05	«有常之逐点», «静有常»	constancy pointwise; identity is constant
L-06	«根», «核为类», «类有常当且仅当含于核»	the root (theorem); the core is a class; a class is constant exactly where it lies inside its core
L-06	«火薪相须», «火之见证不择类»	fire and fuel need each other; the fire witness picks no class
L-06	«两火以薪定理», «两火同火不足以定理»	in two flames the fuel fixes the regularity; in two flames sameness of flame does not suffice
L-06	«定后遍后则核为其内最粗同行之类», «后不定则核可不同行»	successor fixed and everywhere, so the core is the coarsest class inside going alike; successor not fixed, so the core may not go alike

Item	Name	Gloss
L-07	«闭合当且仅当有常»	closure exactly where constant
L-08	«位同则生亡同»，«位同则住同»	same position, same beginning and ending; same position, same abiding
L-08	«位根»，«位核含于核»	the root theorem for the two-way core; the two-way core lies inside the core
L-08	«亡于核下不变»，«生于位核下不变»	ending invariant under the core; beginning invariant under the two-way core
L-08	«位核含于逆核»，«生于逆核下不变»	the two-way core lies inside the converse net's core; beginning invariant under that core
L-08	«生于核下可变之例»，«逆核不含于核之例»	a case of beginning varying under the core; a case of the converse net's core not inside the core
L-09	«延尽不并»，«生守相杂»，«单行则生守不并»	going on and ending exclude each other; generation and keeping mixed; on a single line generation and keeping exclude each other
L-09	«生生即延»，«息即尽»	renewal without end is going on; ceasing is ending
L-09	«尽则非无终»，«尽而续可定则有终»	ends on every branch, so not without an end; and with continuation decidable, an end
L-10	«全三域之周»	the three strong regions cover
L-11	«尽则有终当且仅当排中»	that ending on every branch gives an end is exactly excluded middle
L-11	«后守之定则LPO»，«生成之定则WLPO»，«生成后守之定则LPO»	deciding 后守 gives LPO; deciding 生成 gives WLPO; deciding between them gives LPO
L-12	«核在每层之内»，«可分则不在核»，«恒温与平衡»	the core lies in every stage; told apart, so not in the core; thermostat and balance
L-13	«核则不可致诘»，«致诘不尽»	in the core, so no inquiry tells apart; inquiry does not exhaust
L-13	«不可致诘而有常则混而为一»	told apart by no inquiry, and constant, so merged into one
L-21	«所闭当且仅当同一或及»	within the closure exactly where the same or reached
L-23	«续无外随网»	no outside along the net varies with the net

References

The works cited above, each checked against the publication's record (a library catalogue, the publisher or the DOI) on 24 or 25 October 2026. A section or theorem number was checked against the work itself where noted, otherwise against later work that cites it; where neither was possible it was dropped.

- Yohji Akama, Stefano Berardi, Susumu Hayashi and Ulrich Kohlenbach (2004). An arithmetical hierarchy of the law of excluded middle and related principles. In Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science (LICS 2004), IEEE Computer Society, 192-201.
- Bowen Alpern and Fred B. Schneider (1985). Defining liveness. Information Processing Letters 21(4), 181-185.
- Errett Bishop, Foundations of Constructive Analysis, New York: McGraw-Hill, 1967, xiii+370 pp.
- Patrick Blackburn, Maarten de Rijke and Yde Venema, Modal Logic, Cambridge Tracts in Theoretical Computer Science 53, Cambridge University Press, 2001, xxii+554 pp. (§2.2 "Bisimulations"; Example 1.25, "Bidirectional frames and models", in §1.3, checked against the book's contents and chapter 1)
- Douglas Bridges and Fred Richman, Varieties of Constructive Mathematics, London Mathematical Society Lecture Note Series 97, Cambridge University Press, 1987, x+149 pp., ISBN 0-521-31802-5
- Peter J. Freyd and Andre Scedrov (1990). Categories, Allegories. North-Holland Mathematical Library 39. Amsterdam: North-Holland.
- Anil Gupta (1982). Truth and paradox. Journal of Philosophical Logic 11(1), 1-60.
- Anil Gupta and Nuel Belnap (1993). The Revision Theory of Truth. Cambridge, MA and London: MIT Press (Bradford Books), xii+299 pp. ISBN 0-262-07144-4.
- Matthew Hennessy and Robin Milner, 'Algebraic laws for nondeterminism and concurrency', Journal of the ACM 32(1):137-161, January 1985, doi:10.1145/2455.2460. (Companion: Robin Milner, Communication and Concurrency, Prentice Hall, 1989.)
- Hans G. Herzberger (1982). Notes on naive semantics. Journal of Philosophical Logic 11(1), 61-102. doi:10.1007/BF00302339
- Hajime Ishihara, 'An omniscience principle, the König lemma and the Hahn-Banach theorem', Zeitschrift für mathematische Logik und Grundlagen der Mathematik 36(3):237-240, 1990 (the journal later became Mathematical Logic Quarterly)
- Paris C. Kanellakis and Scott A. Smolka (1990). CCS expressions, finite state processes, and three problems of equivalence. Information and Computation 86(1), 43-68. doi:10.1016/0890-5401(90)90025-D (extended version of PODC 1983, pp. 228-240)
- John G. Kemeny and J. Laurie Snell, Finite Markov Chains, University Series in Undergraduate Mathematics, Princeton NJ: D. Van Nostrand, 1960, viii+210 pp.; 2nd ed. Springer-Verlag 1976 (lumpability: Theorem 6.3.2, as later work cites it; not checked against the book)
- Saul Kripke (1975). Outline of a theory of truth. Journal of Philosophy 72(19), 690-716.
- Kim G. Larsen and Arne Skou, 'Bisimulation through probabilistic testing', Information and Computation 94(1):1-28, 1991, doi:10.1016/0890-5401(91)90030-6 (preliminary report in POPL '89 (16th ACM Symposium on Principles of Programming Languages), 1989, pp. 344-352)
- F. William Lawvere (1969). Diagonal arguments and cartesian closed categories. In Category Theory, Homology Theory and their Applications II, Lecture Notes in Mathematics 92, Springer, 134-145. doi:10.1007/BFb0080769. Reprinted with author commentary in Reprints in Theory and Applications of Categories 15 (2006), 1-13.
- Kurt Lewin (1922). Der Begriff der Genese in Physik, Biologie und Entwicklungsgeschichte: Eine Untersuchung zur vergleichenden Wissenschaftslehre. Berlin: Springer. doi:10.1007/978-3-662-26457-7 (the source of the term genidentity)

- Robin Milner (1989). Communication and Concurrency. Prentice Hall International Series in Computer Science. New York/Hemel Hempstead: Prentice Hall. xi+260 pp. ISBN 0-13-115007-3
- Flavia Padovani (2013). Genidentity and topology of time: Kurt Lewin and Hans Reichenbach. In N. Milkov and V. Peckhaus (eds.), The Berlin Group and the Philosophy of Logical Empiricism, Dordrecht: Springer, 97-122. doi:10.1007/978-94-007-5485-0_5
- Robert Paige and Robert E. Tarjan (1987). Three partition refinement algorithms. SIAM Journal on Computing 16(6), 973-989. doi:10.1137/0216062
- David Park (1981). Concurrency and automata on infinite sequences. In P. Deussen (ed.), Theoretical Computer Science: 5th GI-Conference, Karlsruhe, LNCS 104, Springer, pp. 167-183. doi:10.1007/BFb0017309
- A. N. Prior (1960). The autonomy of ethics. Australasian Journal of Philosophy 38(3), 199-206.
- J. J. M. M. Rutten (2000). Universal coalgebra: a theory of systems. Theoretical Computer Science 249(1), 3-80. doi:10.1016/S0304-3975(00)00056-6
- Gerhard Schurz (1997). The Is-Ought Problem: An Investigation in Philosophical Logic. Trends in Logic (Studia Logica Library) vol. 1. Dordrecht: Kluwer, x+332 pp.
- Amartya Sen (1970). Interpersonal aggregation and partial comparability. Econometrica 38(3), 393-409. doi:10.2307/1909546
- Alfred Tarski (1941). On the calculus of relations. Journal of Symbolic Logic 6(3), 73-89. doi:10.2307/2268577
- Alfred Tarski and Steven Givant (1987). A Formalization of Set Theory without Variables. American Mathematical Society Colloquium Publications, vol. 41. Providence, RI: AMS. xxi+318 pp. ISBN 0-8218-1041-3

Response form

For each item you judge, give one verdict: **accept** (the statement is correctly stated, described and credited as written), **accept with changes** (it stands once rewritten; say how), **reject** (it does not stand; say why) or **cannot judge**. Questions are numbered through the dossier (Q1–Q74), so they can be answered by number; the optional ones are open problems, not judgments asked of you.

A plain email reply is enough: one line per item, for example "L-08: accept with changes. Q21: …", then any longer comments. An item you leave out is read as not judged. The table below is a checklist of the items.

Item	Verdict	Comment
L-01 The calculus and its operations		
L-02 Reach, precedence and co-presence		
L-03 Continuation as a meet		
L-04 The four marks and coarsening		
L-05 Constancy never lets a continuation end		

Item	Verdict	Comment
L-06 The root theorem: the core		
L-07 Closure and bisimulation equivalence		
L-08 The two-way core and positions		
L-09 The map at two grains		
L-10 The strong forms and coverage		
L-11 Excluded-middle calibrations		
L-12 Testing by stages		
L-13 The converse in the grammar		
L-14 The Hennessy–Milner limit and LLPO		
L-15 Settling is LPO		
L-16 Tails: WLPO and LPO as rules		
L-17 Refutation, verification, double negation, Markov		
L-18 Runs, dependent choice, finite branching		
L-19 The pure-core discipline		
L-20 propext as a lumping		
L-21 Expressible is not derived		
L-22 Premises read off one exchange		
L-23 What the dynamics do not entail		
L-24 Widening and positive weightings		
L-25 Lawvere's diagonal as an unfolding		
L-26 The completion criterion K1–K7		
L-27 Revision with an undecided value		
L-28 Curry and contraction		
L-29 The other calibrations		

General. What would an expert in relation algebra, coalgebra, constructive mathematics or modal logic add or object to, beyond the items? Is any classical result restated without credit?

Returning the form. Return your reply to whoever sent you the packet. The terms on which reviews are kept, and whether you are named or quoted, are given in the cover and the covering letter.